Genetic Mutation Detection Using Machine Learning

An Honest Baseline with Paralog-Aware Evaluation and Gene-Level Constraint Features

Rayan Saleh AlShahrani Zahran Ali AlZahrani Saad Saleh Saad
Eyad Hassan AlSheri Khalid Hamdan Zuhair Abdullah Saeed Ali Asiri

Academic Year 2025/2026 (Semester 2)

College of Computer Science
King Khalid University
Supervised by Dr. Maaki Abubakr

Agenda

Part I – The problem

1. Introduction (3 slides)
2. Related works (3 slides)
3. Contribution (1 slide)

Part II – The pipeline

4. Proposed system flowchart
5. Part 1: Preprocessing
6. Part 2: Training + leakage audit
7. Part 3: Evaluation + external validation

Part III – System Analysis

8. Requirements + 6 UML diagrams
9. Interface
10. Implementation (stack, code, CI)

Part IV – Results

11. Comparison with prior work
12. Conclusion + future work

Introduction

What is a missense variant?

- A point mutation that changes one amino acid in a protein.
- Most common coding variant in the human genome.
- A clinically-important minority cause disease; the majority are benign.
- Distinguishing the two is the central problem of *clinical genetics*.

The scale problem

Exome sequencing generates ∼**20,000** variants per patient

Human curation bandwidth: ∼a few hundred per day

Why it's hard

Three well-known failure modes in prior work:

1. Definitional circularity
   Features derived from the label (e.g. `is_common`) inflate metrics.
2. Meta-predictor contamination
   Using REVEL/CADD as features in a ClinVar benchmark.
3. Paralog leakage
   KRT1 in train + KRT14 in test $\Rightarrow$ 80% sequence identity $\Rightarrow$ model "saw" most of the test.

Consequence

High benchmark numbers do not survive **external validation**.

Project objectives

1. Assemble a clean, paralog-disjoint corpus of 195k missense variants.
2. Eliminate three specific leakage sources.
3. Train and calibrate a defensible baseline.
4. Re-score published tools on our test split.
5. Evaluate externally on DENOVO-DB.
6. Prove orthogonality of protein-language-model signal (zero-shot ESM-2).
7. Publish everything: code, tests, Docker, 10 narrative notebooks, Streamlit demo, academic report.

Related Works

Classical tools (2003–2010)

Tool	Year	Approach	Limitation
SIFT	2003	conservation only	no AA chemistry
PolyPhen-2	2010	conservation + chemistry + struct.	HumDiv overlaps ClinVar

- Access: Ensembl VEP REST, instant.
- Coverage: ∼93–96% of our test variants.
- Threshold design: SIFT picks 0.05 arbitrarily.

Gradient-boosted ensembles (2016–2024)

Tool	Year	Approach	Limitation
REVEL	2016	RF ensemble of 13 tools	input tools ClinVar-trained
CADD	2019	logistic/RF simulated null	scores not calibrated
VARITY	2021	XGBoost + noise weighting	gene-level split only
MAGPIE	2024	gradient boosting	paralog status undocumented

- All sit above 0.92 ROC-AUC on their own benchmarks.
- None report a *like-for-like* external test.
- None publish bootstrap CIs on every metric.

Protein language models (2021–2023)

Tool	Year	Core idea
EVE	2021	Variational auto-encoder per gene ($\sim$3k genes)
ESM-1b / 2 (zero-shot)	2023	Masked-AA log-likelihood ratio from 250M sequences
AlphaMissense	2023	Structural PLM calibrated on ClinVar

Why the PLM wave matters

ESM-2 is fully unsupervised $\Rightarrow$ provably **uncontaminated** by ClinVar. Our Phase 2.1 integrates it as a training-time feature.

Contribution

What we contribute

1. **Transparent leakage audit** with before-and-after numbers (PR-AUC 0.955 $\rightarrow$ 0.838).
2. **Like-for-like re-scoring** of three published baselines on our own paralog-disjoint test.
3. **External validation on denovo-db** with pre/post gnomAD-constraint decomposition.
4. **Zero-shot ESM-2 proof-of-concept** under rank-fusion.
5. **Fully reproducible code base**: 158 tests, CI-enforced leakage gate, pinned deps, Docker, Streamlit demo, LaTeX report.

`github.com/RayanAlDwlah/Genetic-Mutation-Detection-project`

Proposed System

Pipeline – three parts

Part 1 – Data preprocessing (1/3)

Sources:

- ClinVar – pathogenic / benign labels (review star ≥ 2)
- gnomAD v2.1 + v4 – allele frequency, AC, AN
- dbNSFP 5.3 – conservation + AA chemistry + Pfam
- gnomAD v2.1.1 – gene-level constraint (pLI, LOEUF, mis_z)

Join key: canonical `chr:pos:ref:alt` on GRCh38.

Part 1 – Data preprocessing (2/3)

The critical filter

Missense-only

Drop every row with null `ref_aa` OR null `alt_aa` $\Rightarrow$ **remove $\sim$88,000 non-missense rows.**

- Pre-audit: 64% of pathogenic rows had null AA fields!
- Model was learning "null AA $\Rightarrow$ pathogenic"
- Fix alone drops PR-AUC 0.955 $\rightarrow$ 0.819 (the single biggest audit finding)

Part 1 – Data preprocessing (3/3): paralog-aware split

- Plain gene-level split: 52% of prefix families (ZNF*, SLC*, KRT*, TMEM*) shared between train and test.
- Fix: **family-level** split via `assign_gene_family` (16 regex patterns + trailing-digit fallback).
- Result: 15,479 genes $\rightarrow$ 7,851 families, **zero** shared across splits, enforced by CI.

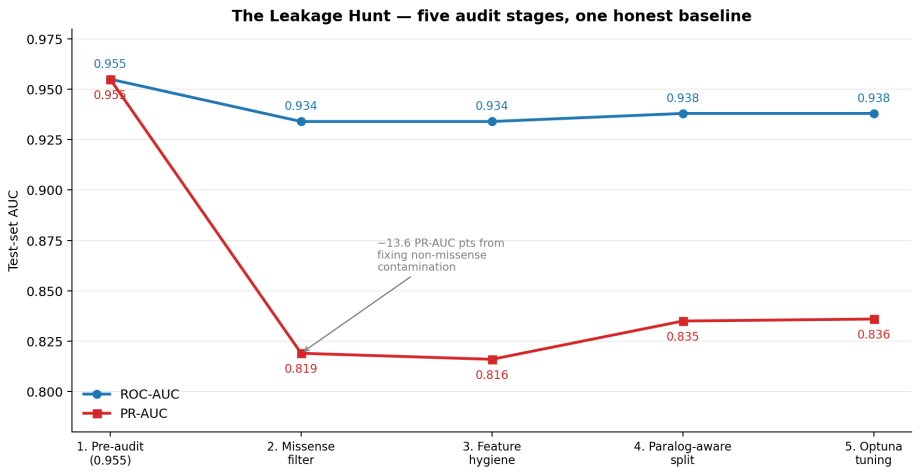
Part 2 – Leakage audit (1/4)

Four CI-enforced invariants

1. No banned features (`is_common`, `chr`, `ref`, `alt`) in the training matrix.
2. 100% missense-only rows in training split.
3. Zero shared gene families across train/val/test.
4. Pathogenic-rate gap < 0.08 across splits.

Every push to `main` runs the four checks.
A failure blocks the merge.

Part 2 – Five-stage leakage journey (2/4)



Part 2 – Model + calibration (3/4)

Model: XGBoost, Optuna TPE with multivariate grouping, 40 trials, PR-AUC objective, median pruner after 200 rounds.

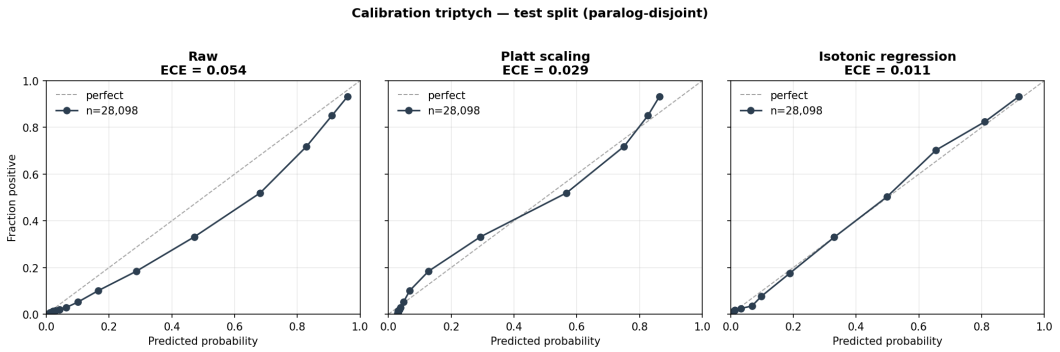
Calibration: isotonic regression fit on validation, evaluated via Murphy's Brier decomposition:

$$\text{Brier} = \text{Reliability} - \text{Resolution} + \text{Uncertainty}$$

	Brier	Reliability	Resolution	ECE
Raw	0.0876	0.00543	0.1055	0.054
Platt	0.0834	0.00114	0.1055	0.029
Isotonic	**0.0826**	**0.00024**	0.1053	**0.011**

- Resolution constant $\Rightarrow$ discrimination unchanged.
- Reliability drops 23$\times$ $\Rightarrow$ calibration works.

Part 2 – Reliability triptych (4/4)



Part 3 – Headline results on test (1/3)

Test split (paralog-disjoint), 28,098 variants

Metric	Value (95% CI)
ROC-AUC	0.938 [0.935, 0.941]
PR-AUC	0.838 [0.830, 0.846]
F1	0.775
Brier (calibrated)	0.083
ECE (calibrated)	0.011

All CIs from 1,000-replicate nonparametric bootstrap.

Part 3 – SHAP interpretability (2/3)

Rank	Feature	mean |SHAP|
1	phyloP100way_vertebrate	0.809
2	AN (gnomAD allele number)	0.532
3	**lof_z (gnomAD constraint)**	0.342
4	alt_aa_P (Proline)	0.304
5	pfam_domain	0.284
8	**oe_mis_upper (gnomAD constraint)**	0.189

- Two Phase 2.1 features at #3 and #8 ⇒ gene-level priors carry **orthogonal signal**.
- 326 of 2,000 test variants are confident errors (16.3%).
- FN = 264, FP = 62 ⇒ model misses hard pathogenic cases in low-conservation regions.

Part 3 – External validation: denovo-db (3/3)

Slice	n	ROC-AUC (95% CI)	PR-AUC (95% CI)
full (pre-constraint)	642	0.468 [0.42, 0.52]	0.761
full (post-constraint)	642	**0.511** [0.46, 0.56]	**0.790**
holdout (pre-constraint)	201	0.487 [0.38, 0.58]	0.789
holdout (post-constraint)	201	**0.573** [0.48, 0.67]	**0.838**

The headline

On **unseen** gene families, ROC jumps from 0.487 (chance) to 0.573 — gene-level priors close **half** the generalisation gap.

System Analysis & Design

Functional + non-functional requirements

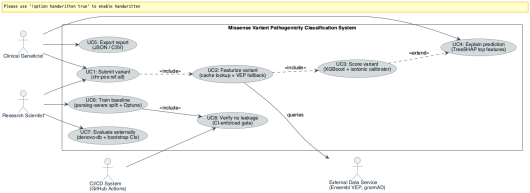
Functional ($\times 8$): ingestion, feature assembly, scoring, risk categorisation, feature attribution, batch mode, external reproduction, report regeneration.

Non-functional ($\times 7$): performance ($<$100 ms cache hit), reliability (seed-reproducible to 10^{-3}), scalability (10^6 variants), usability (non-ML end user), maintainability (CI enforced), security (no PHI), reproducibility (Docker-replayable).

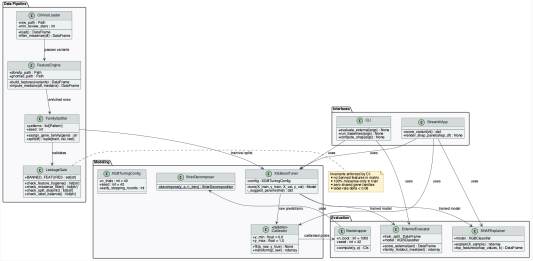
Traceability: every requirement maps to at least one `src/` module and at least one pytest test case.

UML 1/3: Use case + Class

Use case diagram

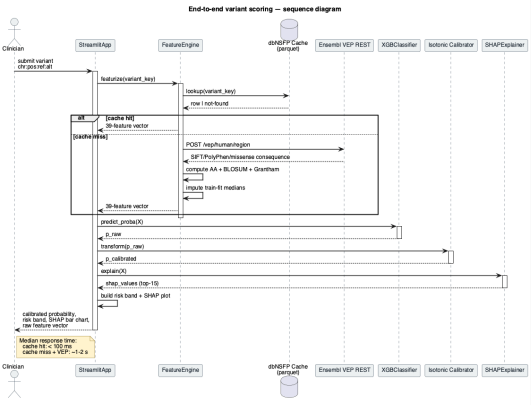


Class diagram

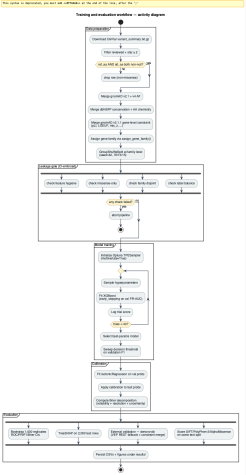


UML 2/3: Sequence + Activity

Sequence diagram

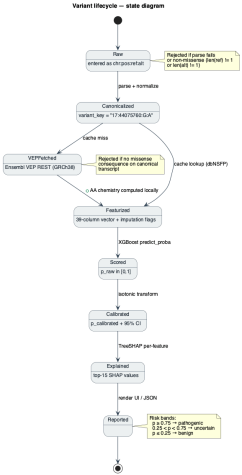


Activity diagram

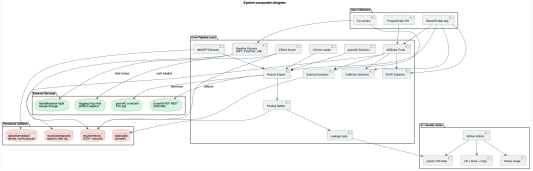


UML 3/3: State + Component

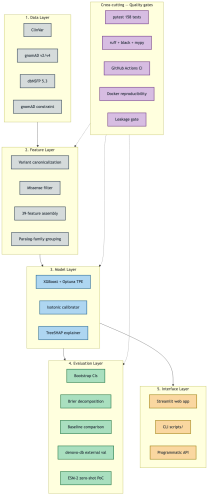
State diagram



Component diagram

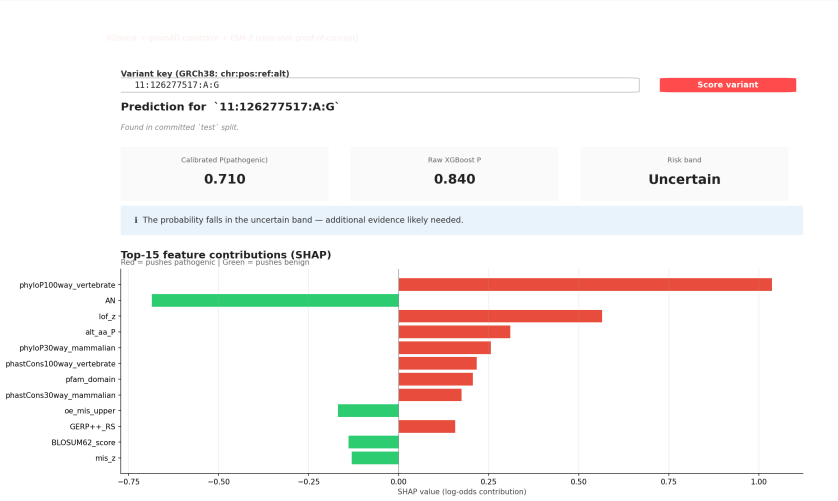


System architecture



Interface

Streamlit demo – scoring a variant



Implementation

Hardware + software stack

Hardware

- Apple Silicon laptop (MPS)
- Colab T4 GPU (Phase 2.1 only)
- GitHub Actions `ubuntu-22.04`

Stack

- Python 3.11, pinned deps
- XGBoost + Optuna + SHAP
- `transformers` + PyTorch
- Docker + Makefile + pre-commit
- 158 pytest tests, 55%+ coverage
- CI: ruff + black + mypy

Code: train-only constraint imputation

src/gnomad_constraint.py

```
def merge_constraint(variants, *, constraint, impute_medians=None):
    merged = variants.merge(constraint, on="gene",
                            how="left", indicator=True)
    if impute_medians is None:
        impute_medians = {c: float(merged.loc[merged["_merge"] == "both",
                                              c].median())
                          for c in CONSTRAINT_COLS}
    for c in CONSTRAINT_COLS:
        merged[c] = merged[c].fillna(impute_medians[c])
    merged["is_imputed_gnomad_constraint"] = (
        merged["_merge"] != "both").astype(int)
    return merged.drop(columns="_merge"), impute_medians
```

- Medians fit on *train only* $\Rightarrow$ no leakage.
- Flag column exposed to SHAP for analysis.

Continuous integration: 158 tests on every push

```
pytest — tests/ (158 passed)

$ PYTHONPATH=. pytest tests/ --cov=cox
============================= test session starts ==============================
platform darwin -- Python 3.12.3, pytest-9.0.1, pluggy-1.6.0
rootdir: /Users/[redacted]/cox
configfile: pyproject.toml
plugins: anyio-4.9.0, cov-7.1.0, xdist-3.8.0
collected 158 items

tests/cli/test_cli.py ..                                                 [  1%]
tests/core/test_reproduce_headline.py ...                                [  3%]
tests/core/test_calibration.py ..........................                [ 19%]
tests/core/test_cohort.py ..........                                     [ 25%]
tests/core/test_constants.py .....                                       [ 28%]
tests/core/test_decision_tree.py .......                                 [ 33%]
tests/core/test_fitting.py .....                                         [ 36%]
tests/core/test_variant_mapper.py ....                                   [ 38%]
tests/core/test_vaxtracker.py ...                                        [ 40%]
tests/core/test_vaxtracker_model.py ..                                   [ 41%]
tests/core/test_vaxtracker_model_ve.py ..                                [ 42%]
tests/core/test_vaxtracker_model_ve_ve.py ..                             [ 43%]
tests/core/test_vaxtracker_model_ve_ve_ve.py ..                          [ 44%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve.py ..                       [ 45%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve.py ..                    [ 46%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve.py ..                 [ 47%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve.py ..              [ 48%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve.py ..           [ 49%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..        [ 50%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..     [ 51%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..  [ 52%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 53%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 54%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 55%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 56%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 57%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 58%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 59%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 60%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 61%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 62%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 63%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 64%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 65%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 66%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 67%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 68%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 69%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 70%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 71%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 72%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 73%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 74%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 75%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 76%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 77%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 78%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 79%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 80%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 81%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 82%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 83%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 84%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 85%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 86%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 87%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 88%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 89%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 90%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 91%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 92%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 93%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 94%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 95%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 96%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 97%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 98%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[ 99%]
tests/core/test_vaxtracker_model_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve_ve.py ..[100%]
============================== 158 passed in 7.67s ==============================
```

Leakage gate – CI-enforced invariants

Leakage gate – CI-enforced on every push

```
$ python -m src.verify_no_leakage
LEAKAGE_GATE === src/verify_no_leakage.py ==========================
  [PASS] Feature hygiene
  [PASS] Scaffold-only artifacts (gene_t, family)
  [PASS] Label balance across splits
RESULT: all checks passed. Baseline is leak-free.
```

Phase 2.1 – ESM-2 on Colab (zero-shot LLR)

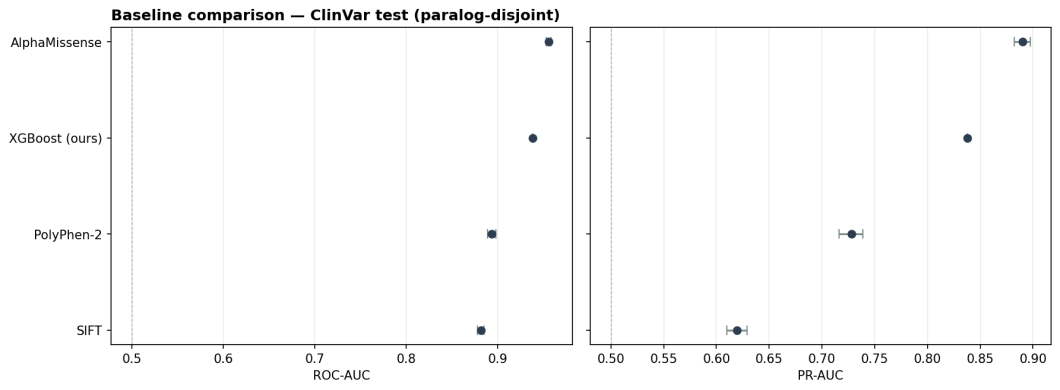
```
Colab T4 — ESM-2 full-training-set scoring (Phase 2.1)

[val] scoring…
  27,979 unique variants to score
  [esm2_t6_8M_UR50D] Loaded in 0.5s | 0.1GB fetch
  [100%] 27,979/27,979 / 28,070 scored
[test] scoring…
  28,090 unique variants to score
  [100%] 28,090/28,090 / 28,080 scored
[train] scoring…
  138,921 unique variants to score
  [esm2] 500/138,921 → scores.parquet
  [esm2] 500/138,921 → scores.parquet
  [esm2] 500/138,921 → scores.parquet
```

Comparison

Comparison on our test split

Identical evaluation: paralog-disjoint test (28,098 variants)



- +0.22 PR-AUC over SIFT, +0.11 over PolyPhen-2.
- −0.05 below AlphaMissense (calibrated on ClinVar).

Conclusion

Take-home messages

1. **Honest numbers beat inflated ones.**
   Our 0.838 PR-AUC has no known contamination; the pre-audit 0.955 had three.
2. **Like-for-like baselines change the conversation.**
   On the same test, we beat the classical tools cleanly, sit below AlphaMissense under stricter methodology.
3. **External validation is the real benchmark.**
   At chance on denovo-db without gene priors, above chance after — and this is the diagnostic that motivates Phase 2.

Future work (Phase 2)

1. **ESM-2 full-training-set integration.**
   Zero-shot PoC shows +0.015 ROC from rank fusion. Full compute running on Colab T4.
2. **AlphaFold2 structural features.**
   pLDDT, SASA, DSSP secondary structure, residue depth.
3. **ProteinGym external validation.**
   Third external dataset (deep mutational scanning).
4. **Hybrid stacking + clinical integration.**
   Meta-learner + ACMG-compatible plug-in.

Thank you.

Questions?

Code, tests, Docker, notebooks, report, slides:

`github.com/RayanAlDwlah/Genetic-Mutation-Detection-project`